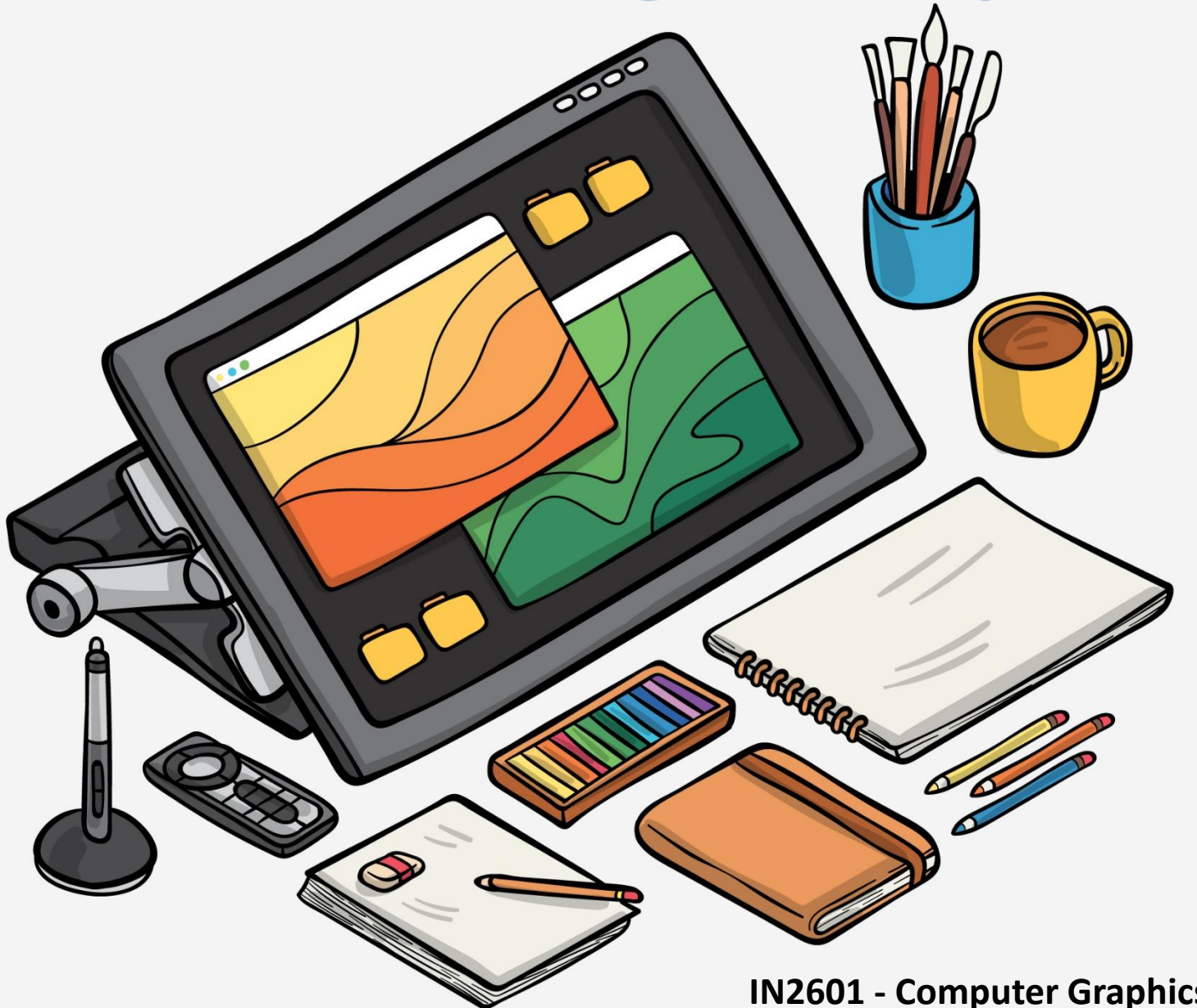


Filling 2D Computer Graphics



W.A.G.Nilnuwani
Department of Information Technology,
Faculty of Information Technology

Learning Outcomes

- By the end of this lesson, you should be able to,
 - define polygon filling in computer graphics
 - explain why filling algorithms are needed
 - describe the working of scan-line, boundary fill, and flood fill algorithms
 - distinguish between boundary fill and flood fill
 - apply polygon fill algorithms to simple examples
 - solve exercise questions related to polygon filling

Introduction to Region Filling

- **Region filling** is the process of coloring or shading a selected area in an image.
- It is used in computer graphics to
 - fill shapes with color
 - highlight selected regions
 - display solid objects
 - support painting and drawing applications
- A **region** is an enclosed area bounded by edges or boundary pixels.

Definition of a Region

a) Pixel-level definition

- At the pixel level, a region can be described as,
 - the set of boundary pixels that outline it, or
 - the complete set of pixels that belong to the area

b) Geometric-level definition

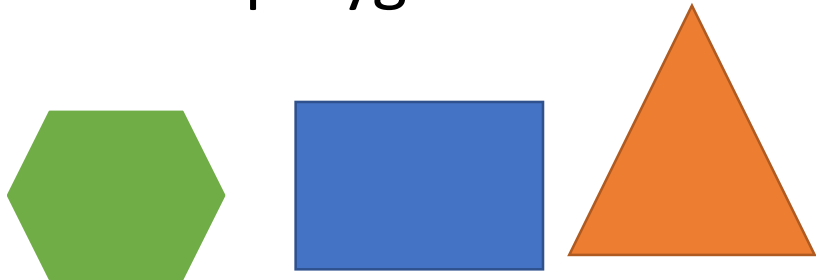
- At the geometric level, a region is described using shapes such as
 - polygons
 - circles
 - curves
 - other closed figures

Types of Polygon

- A polygon is a closed figure made up of straight line segments.
- Polygons are commonly used to represent regions in computer graphics.

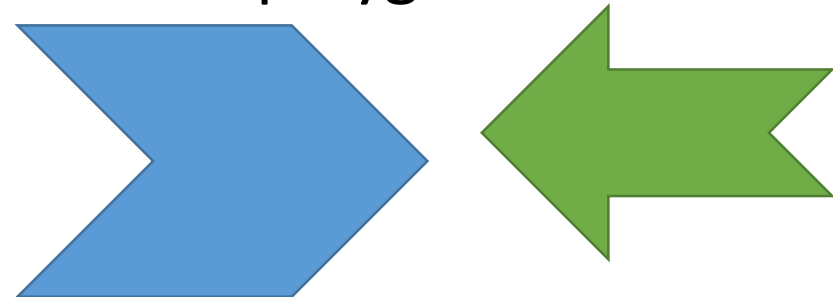
- Convex Polygon

A convex polygon is a polygon in which the line segment joining any two points inside the polygon lies completely inside the polygon.



- Concave Polygon

A concave polygon is a polygon in which the line segment joining two points inside the polygon may pass outside the polygon.



What is a Polygon?

- **In computer graphics**

- A polygon is represented using a set of vertices,
 $(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots$

Introduction to Polygon Filling

- **What is polygon filling?**

- Polygon filling is the process of coloring the interior region of a closed polygon.

- **Why is it important?**

- makes shapes appear solid
- used in painting, CAD, games, animation, GUI design
- helps render objects realistically

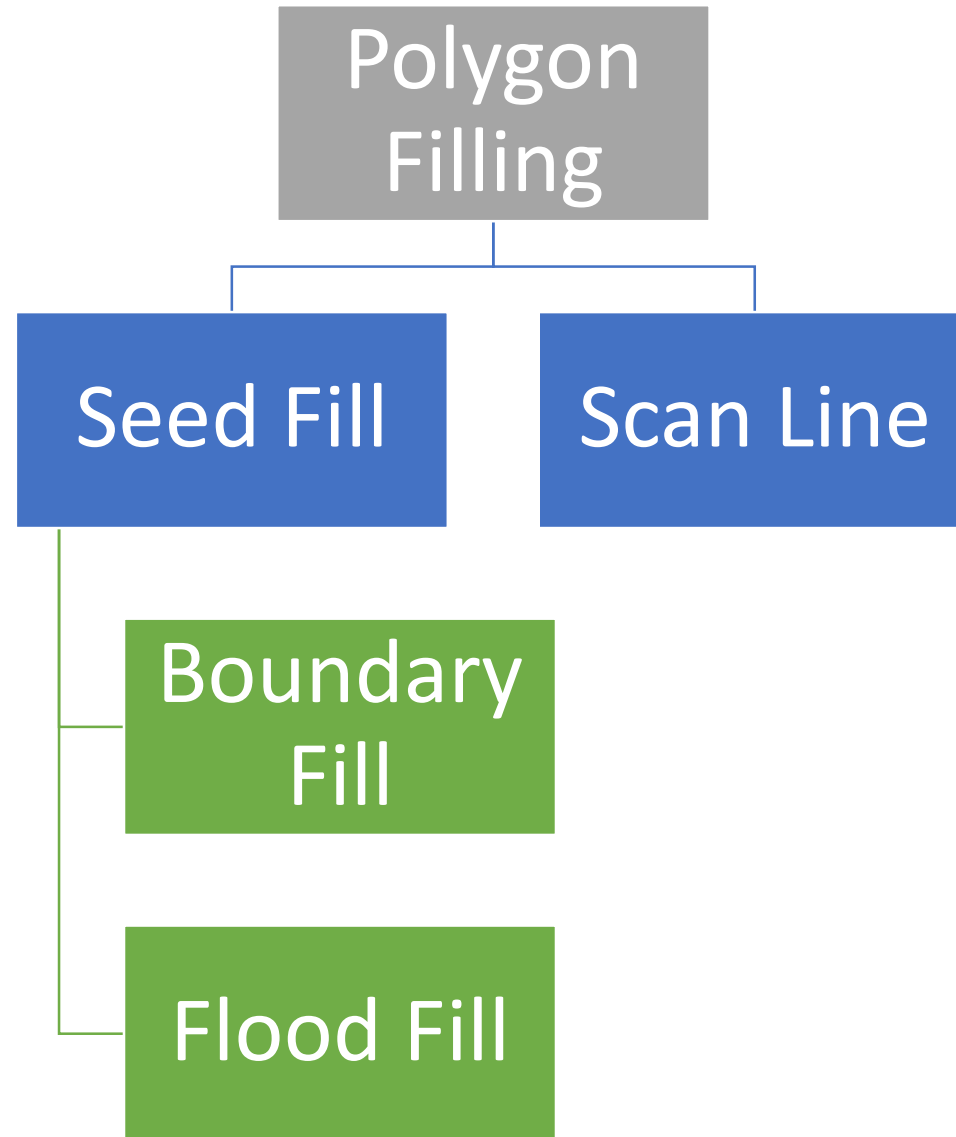
- **Example**

- outline of triangle - only edges visible
- filled triangle - complete solid shape visible

Need for Polygon Filling

- **Why not just draw boundaries?**
 - Drawing only edges gives a hollow shape.
- **Why fill polygons?**
 - to assign colors to objects
 - to show surfaces
 - to represent interiors of shapes
 - to improve visual quality
- **Real applications**
 - paint software
 - map coloring
 - 2D games
 - 3D object rendering

Main Approaches of Polygon Filling



Scan-Line Fill Algorithm

- **Basic concept**

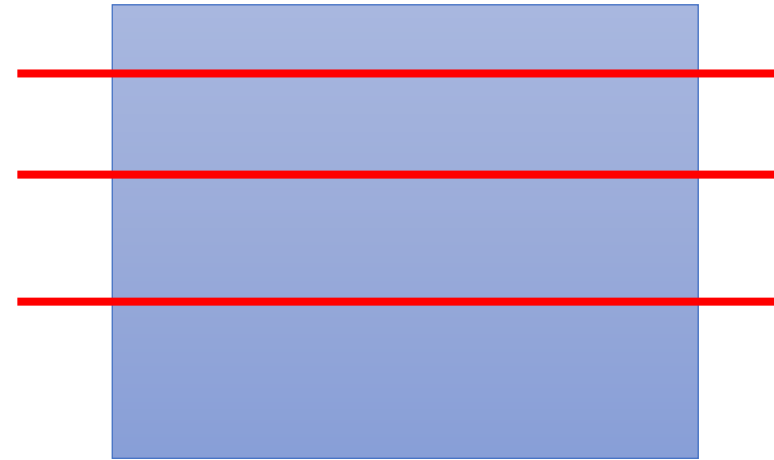
- A horizontal line is moved across the polygon from bottom to top.
- For each scan line,
 - find where it intersects polygon edges
 - sort the intersection points
 - fill pixels between intersection pairs inside the polygon.

- **Key idea**

- If a scan line enters the polygon, it must later leave it.

- So,

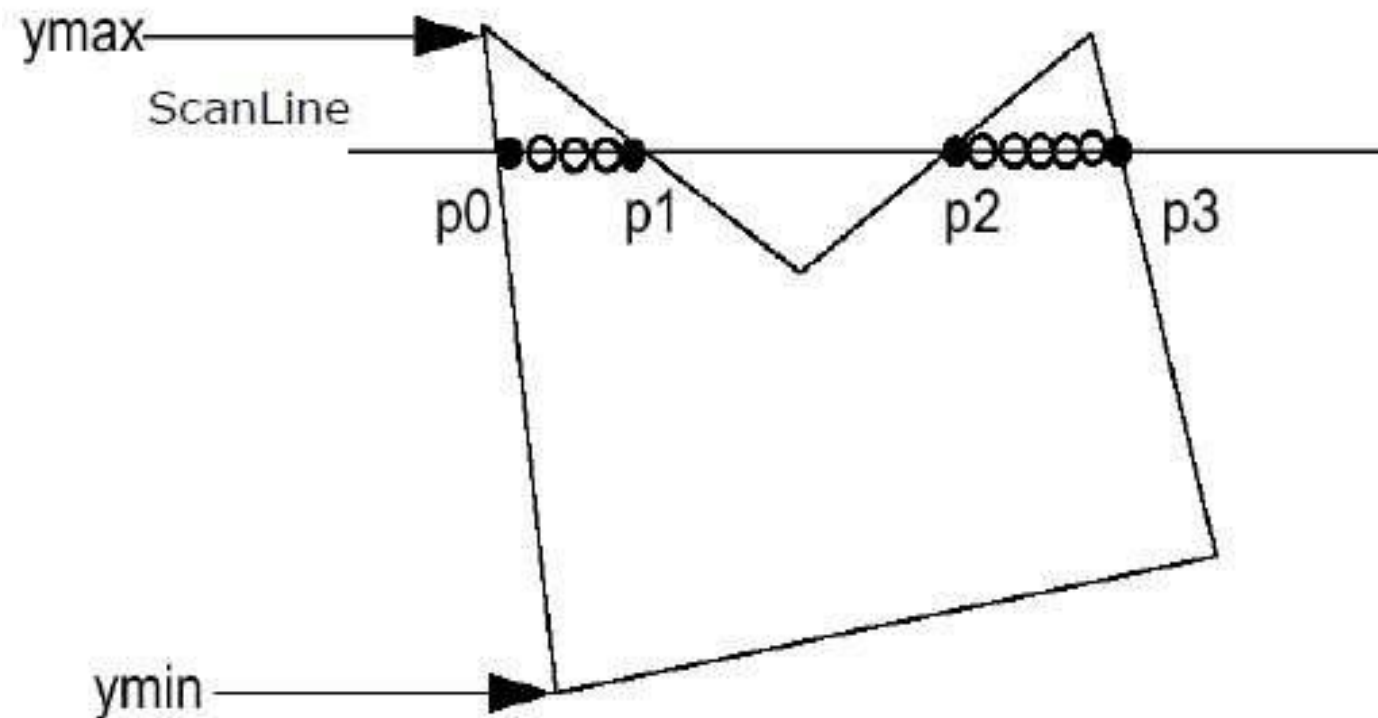
- fill between 1st and 2nd intersection
- fill between 3rd and 4th intersection



Scan-Line Fill Algorithm Cont...

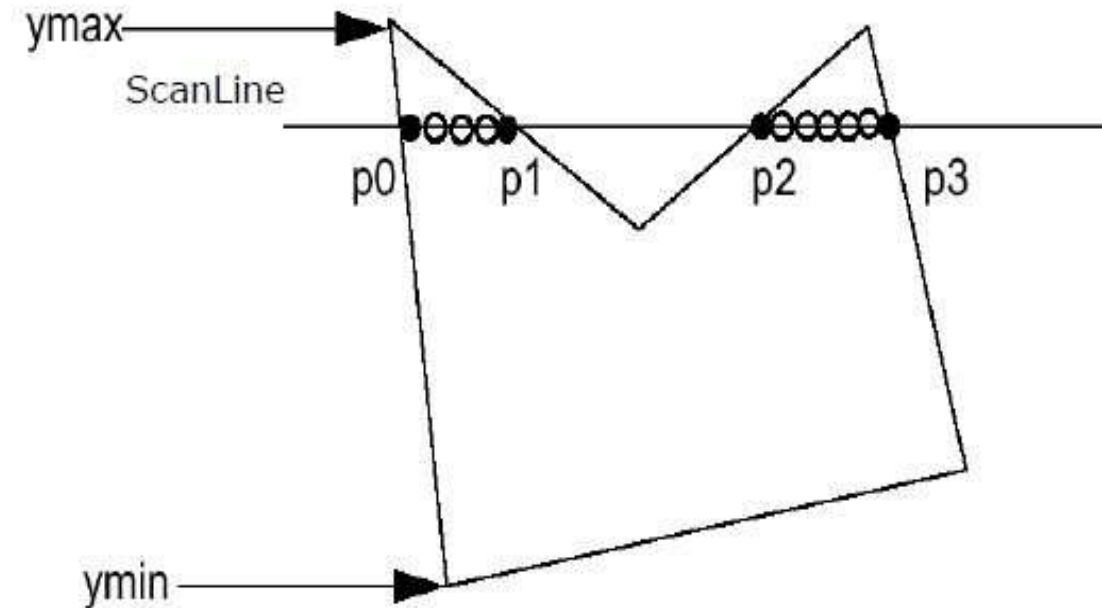
- Step 1 - Find Y_{min} and Y_{max}
 - First, determine
 - Y_{min} - the lowest y-coordinate of the polygon
 - Y_{max} - the highest y-coordinate of the polygon
- Step 2 - Find Intersection Points
 - For each scan line between Y_{min} and Y_{max}
 - check where the scan line intersects the polygon edges
 - mark those points of intersection
- Step 3 - Sort Intersection Points
 - The intersection points must be arranged in increasing order of x-coordinate
- Step 4 - Fill Between Pairs
 - fills the regions that are inside the polygon and ignores alternate outside regions.

Scan-Line Fill Algorithm Cont...



Scan-Line Fill Algorithm Cont...

- Identify Y_{min} and Y_{max}
- The scan line intersects the polygon at p_0 , p_1 , p_2 , p_3
- These are the x-coordinate crossing points on that scan line.
- increasing order of x-coordinate - P_0, P_1, P_2, P_3 (filling must be done from left to right between valid pairs.)
- Pairs are $(P_0, P_1), (P_2, P_3)$
- So the algorithm fills,
 - from p_0 to p_1
 - from p_2 to p_3

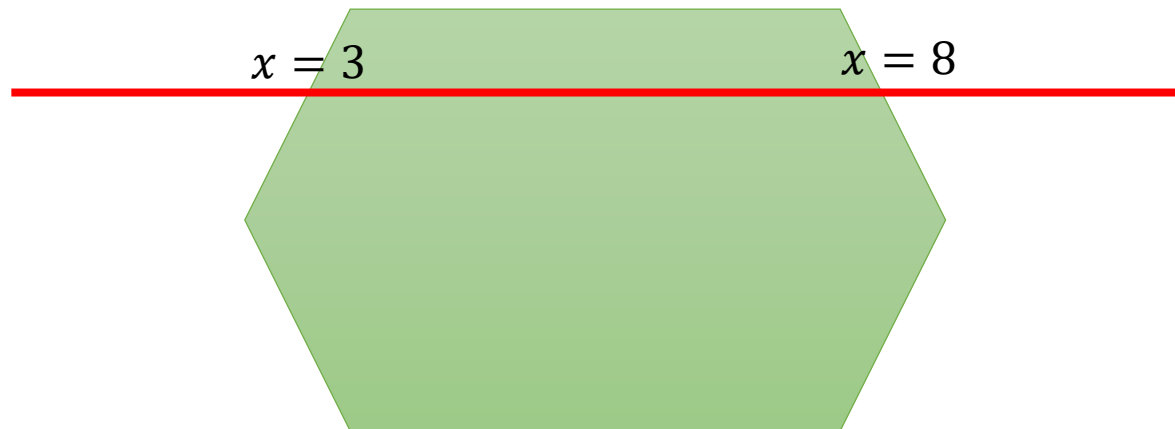


Rules Used in Scan-Line Polygon Filling

- In the scan-line polygon fill algorithm, the scan line may intersect polygon edges in different ways.
 - To decide which pixels should be filled correctly, we follow a set of rules.
1. Intersection Rule
 2. Sorting Rule
 3. Odd-Even Rule
 4. Vertex Handling Rule
 5. Horizontal Edge Handling Rule

Intersection Rule

- For each scan line, find all the points where the scan line intersects the edges of the polygon.
- These intersection points show where the scan line enters and leaves the polygon.
- If a scan line cuts the polygon at $x=3$, 8 then the line enters at $x=3$ and leaves at $x=8$



Sorting Rule

- All intersection points on a scan line must be sorted from left to right in increasing order of x-coordinate.
- Without sorting, we cannot correctly determine which regions are inside the polygon.

Odd-Even Rule

- After sorting the intersections,
 - fill between the 1st and 2nd intersection
 - skip between the 2nd and 3rd
 - fill between the 3rd and 4th
 - continue alternately
- If intersections are
 - 2, 5, 8, 11
- then fill
 - from 2 to 5
 - from 8 to 11

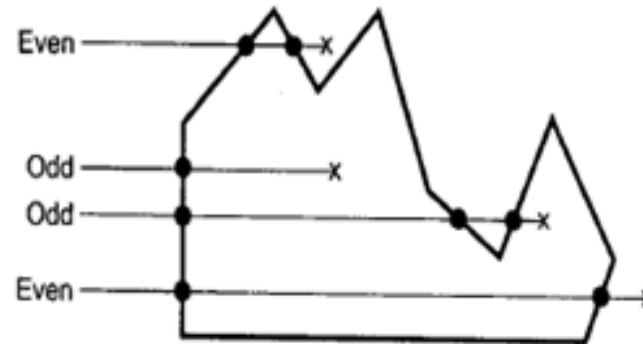


Fig. (a)

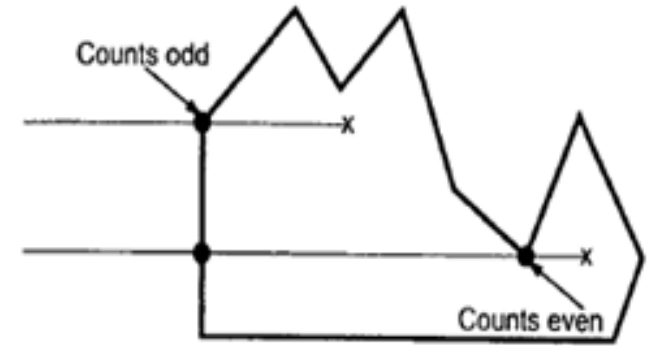


Fig. (b)

Vertex Handling Rule

- When a scan line passes exactly through a polygon vertex, special care is needed.
- Because two edges meet at the same vertex, and the same point may be counted twice.
 - include lower endpoints
 - exclude upper endpoints
- If a scan line passes through a top vertex, we count it only once, not twice.

Horizontal Edge Handling Rule

- Horizontal edges are usually ignored in intersection counting or handled separately.
- Because a horizontal edge can create ambiguity.
 - it may produce multiple points on the same scan line
 - it may disturb the odd-even filling logic

Example - Filling a Rectangle



- $A(2,2)$, $B(8,2)$, $C(8,6)$, $D(2,6)$
- $Y_{min} = 2$ and $Y_{max} = 6$
- For each scan line from $y = 2$ to $y = 6$:
 - left intersection = $x = 2$
 - right intersection = $x = 8$

Filling

$y = 2 \rightarrow$ fill $x = 2$ to 8

$y = 3 \rightarrow$ fill $x = 2$ to 8

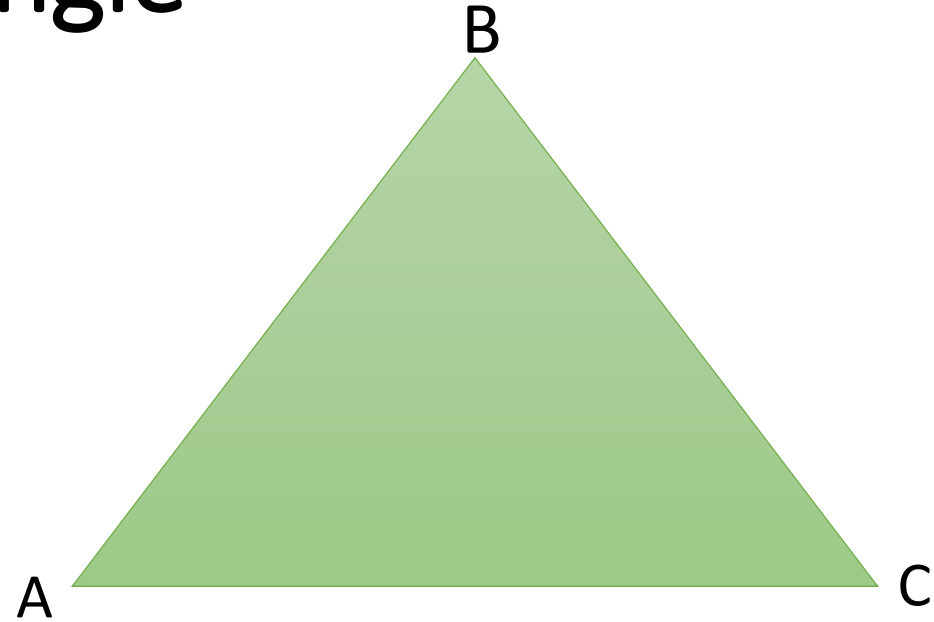
$y = 4 \rightarrow$ fill $x = 2$ to 8

$y = 5 \rightarrow$ fill $x = 2$ to 8

$y = 6 \rightarrow$ fill $x = 2$ to 8

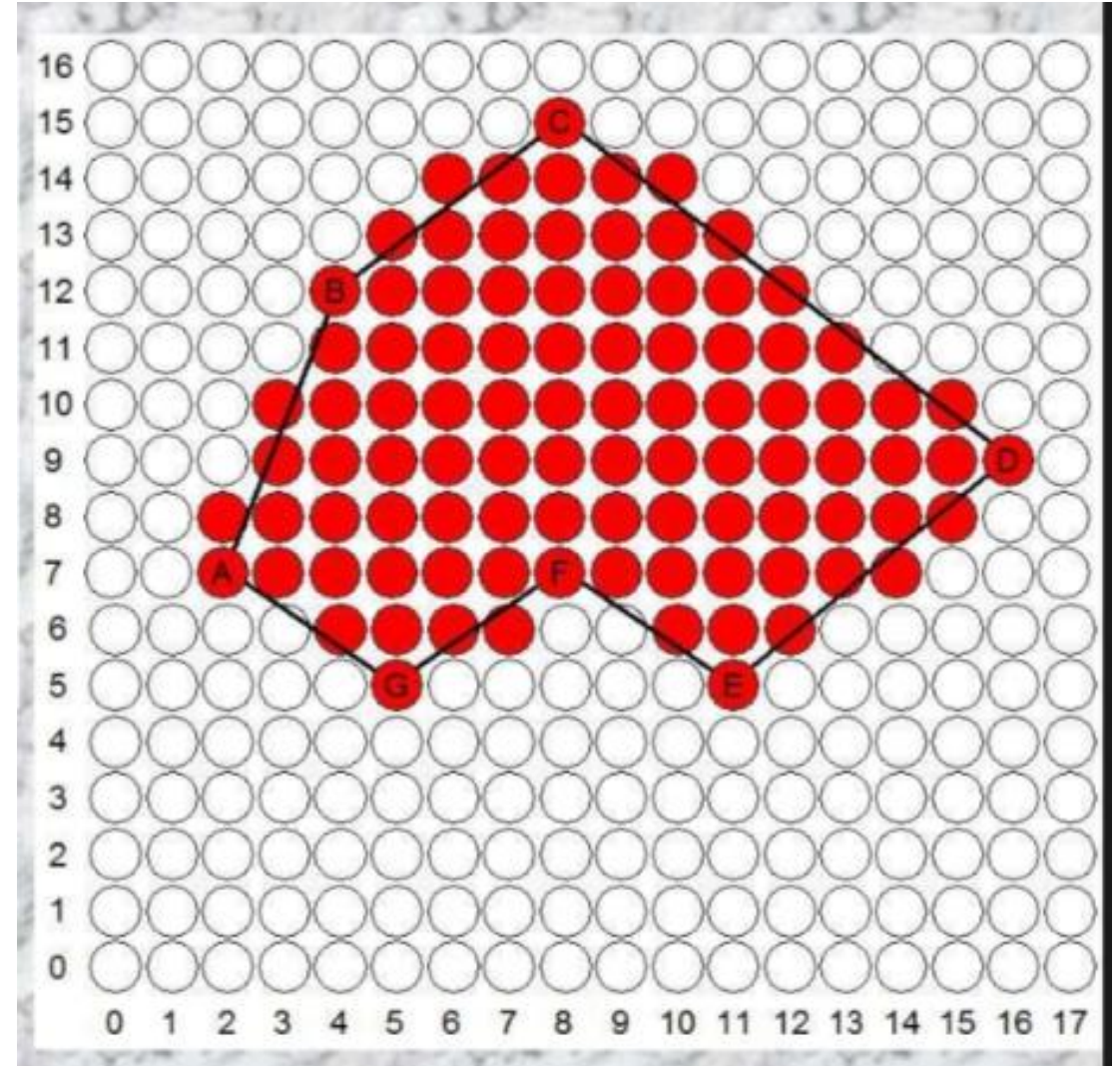
Example - Filling a Triangle

- Triangle vertices
- $A(2,2)$, $B(6,6)$, $C(10,2)$
- $Y_{min} = 2$ and $Y_{max} = 6$
- Intersections
 - $y = 2 \rightarrow x = 2$ and $x = 10$
 - $y = 3 \rightarrow x = 3$ and $x = 9$
 - $y = 4 \rightarrow x = 4$ and $x = 8$
 - $y = 5 \rightarrow x = 5$ and $x = 7$
 - $y = 6 \rightarrow x = 6$
- Filling
 - fill between each pair of x-values



Exercise

- Apply the Scan line polygon filling algorithm and rules for each scan line.



Boundary Fill Algorithm

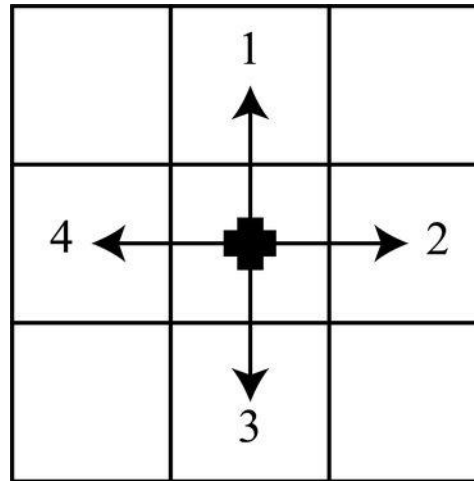
- Start from a point inside the polygon and fill outward until the boundary color is reached.
- Useful when a shape has a clear boundary.
- **Steps**
 1. Start from an interior pixel
 2. Check whether it is boundary color or fill color
 - If not, fill it
 3. Repeat for neighboring pixels
 4. Stops when
 - The fill reaches the boundary color

Flood Fill Algorithm

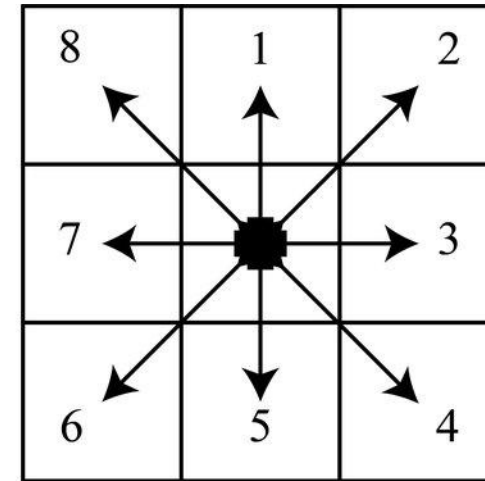
- Start from a seed point and replace all connected pixels having the same old color with a new color.
- Flood fill does not depend on boundary color. It depends on the existing region color.
- **Steps**
 1. Start from a seed pixel
 2. check whether its color is the target old color
 3. if yes, replace it with new color
 4. recursively fill neighboring pixels of the same old color

Connectivity

- Boundary-fill and Flood-fill algorithm can be implemented using 4-connected pixels and 8-connected pixels.
1. **4-connected pixels** : Neighboring pixels are pixels on left, right, above and below of current pixel.
 2. **8-connected pixels** : Neighboring pixels are left, right ,above ,below and 4 diagonal pixels.



4-Connected



8-Connected

4-Connected Boundary Fill

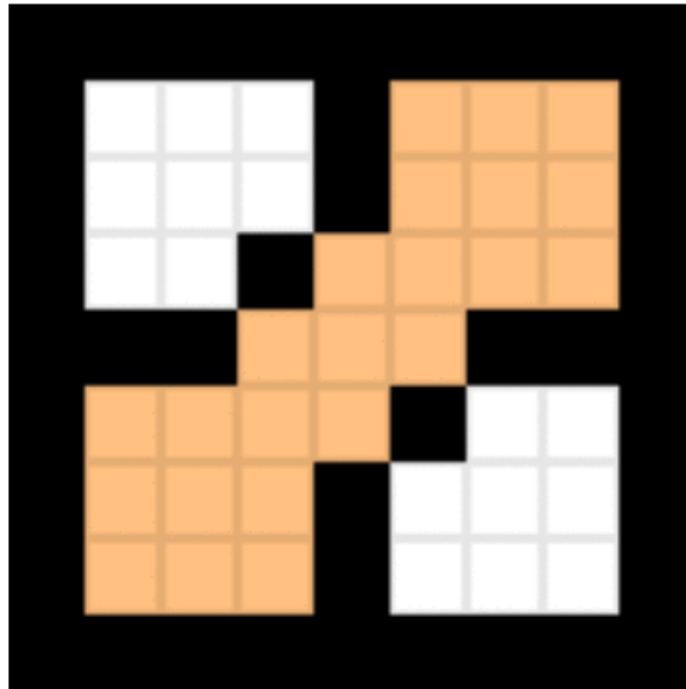
- **Neighbors checked**

- right: $(x+1, y)$
- left: $(x-1, y)$
- up: $(x, y+1)$
- down: $(x, y-1)$

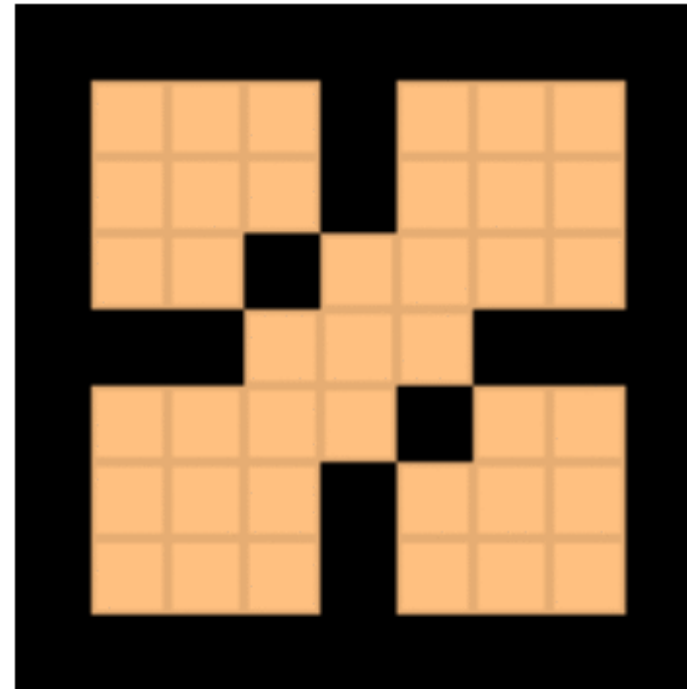
8-Connected Boundary Fill

- **Neighbors checked**
- Includes 4-connected neighbors plus diagonals,
 - right: $(x+1, y)$
 - left: $(x-1, y)$
 - top: $(x, y+1)$
 - bottom: $(x, y-1)$
 - top right: $(x+1, y+1)$
 - top left: $(x-1, y+1)$
 - bottom right: $(x+1, y-1)$
 - bottom left: $(x-1, y-1)$

Example

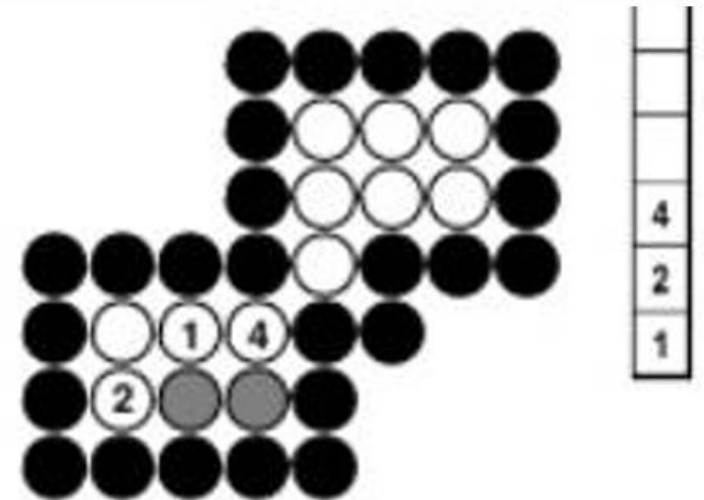
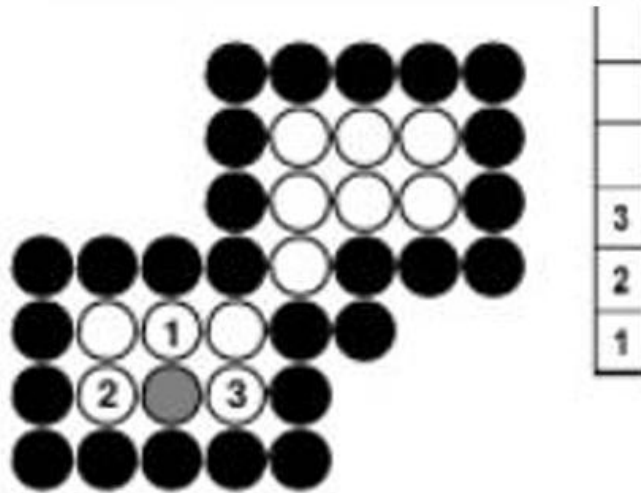
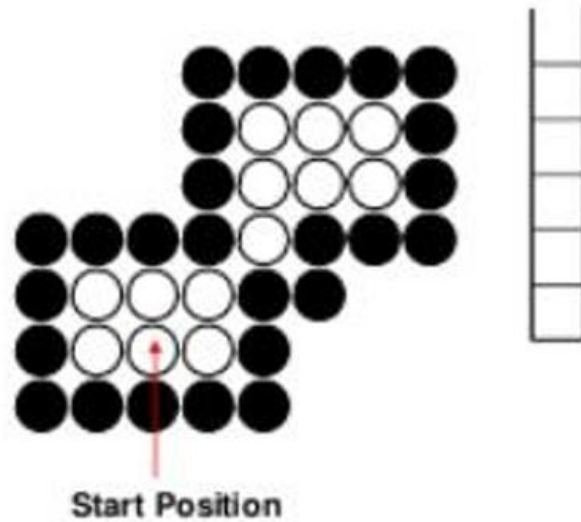


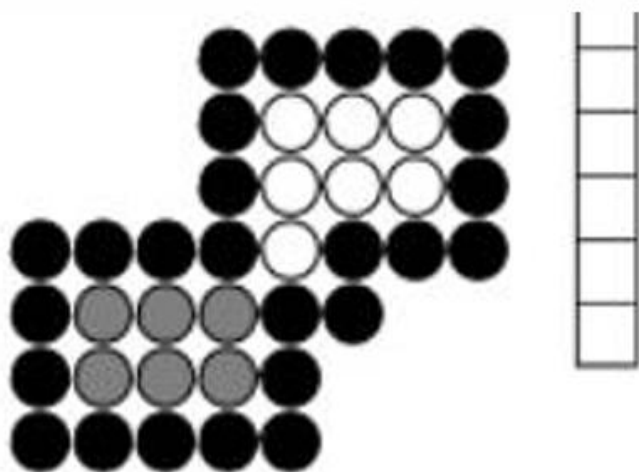
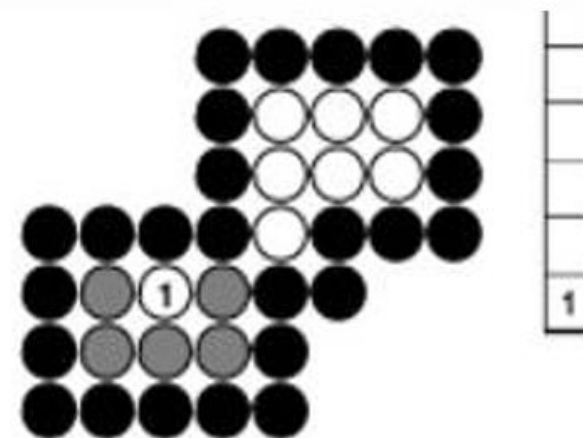
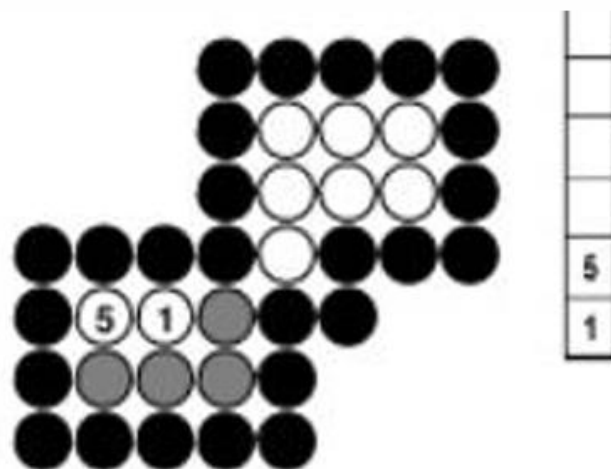
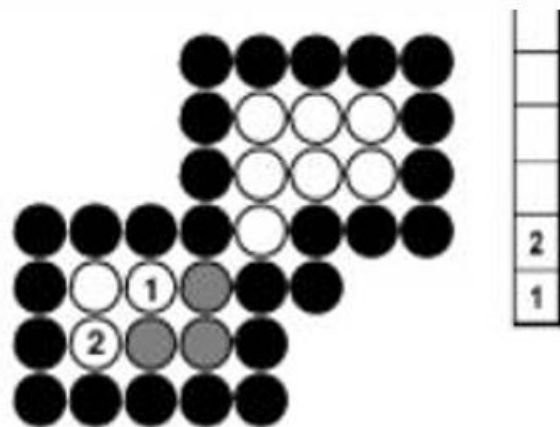
4 Connected



8 Connected

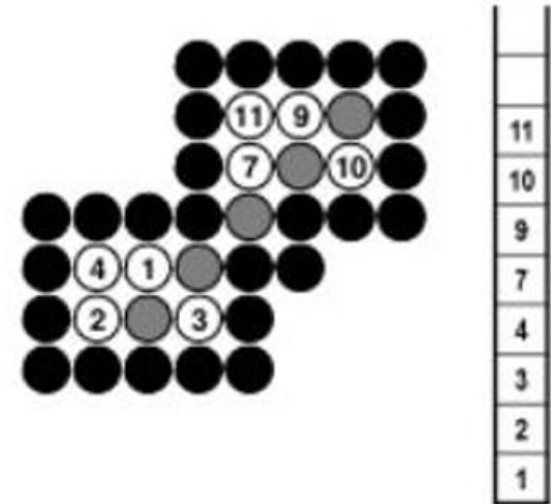
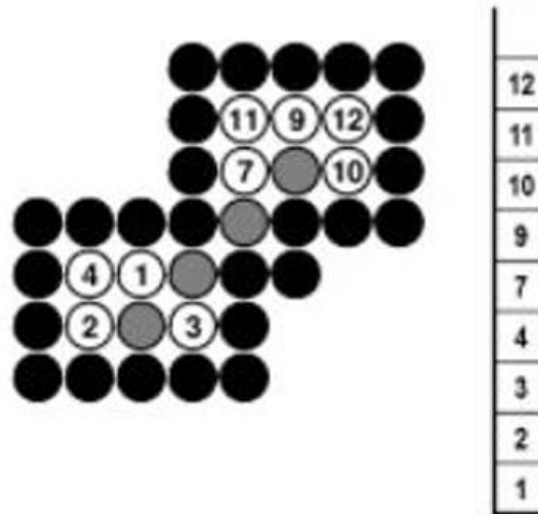
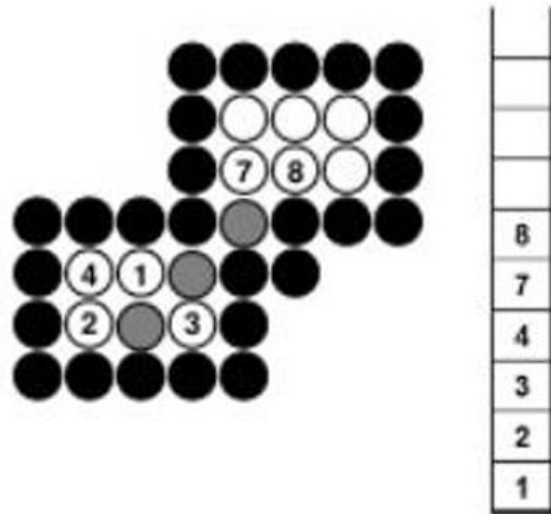
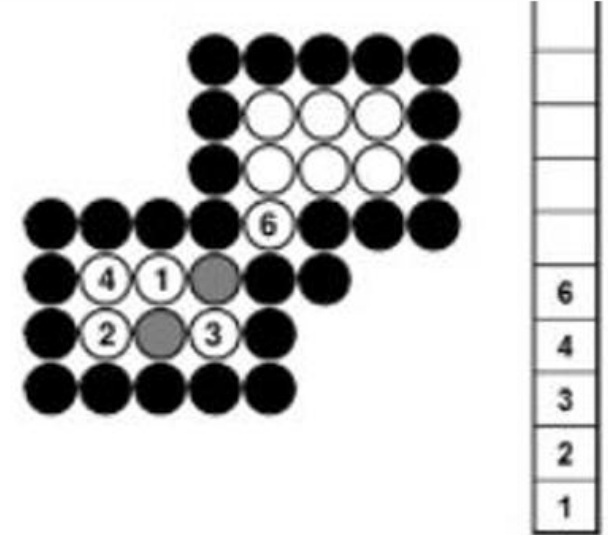
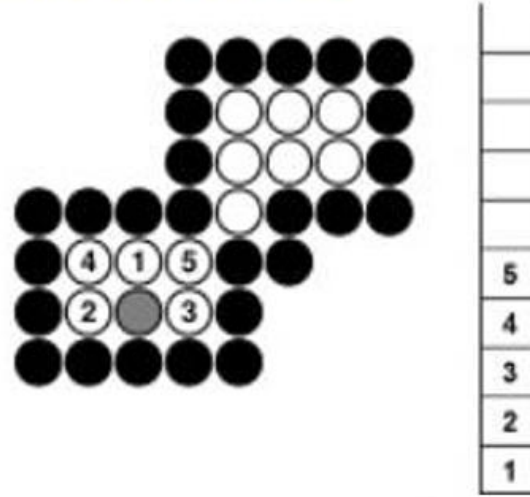
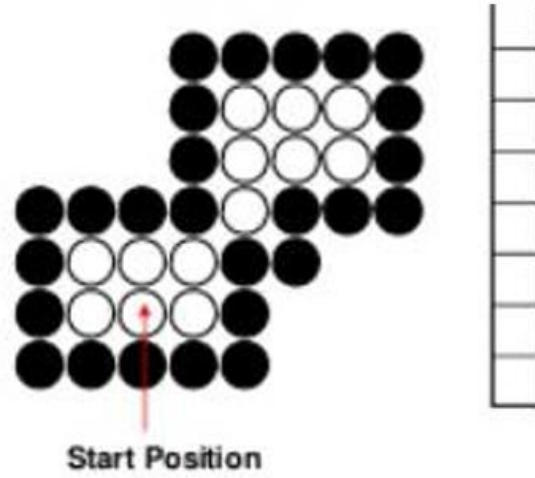
4 – Connected Example





- The 4-connected pixel technique failed to fill the upper area of the polygon

8 – Connected Example



4-Connected Boundary Fill

```
BoundaryFill4(x, y, fillColor, boundaryColor)
{
    if getPixel(x, y) != boundaryColor AND getPixel(x, y) != fillColor
    {
        setPixel(x, y, fillColor)

        BoundaryFill4(x + 1, y, fillColor, boundaryColor) // right
        BoundaryFill4(x - 1, y, fillColor, boundaryColor) // left
        BoundaryFill4(x, y + 1, fillColor, boundaryColor) // up
        BoundaryFill4(x, y - 1, fillColor, boundaryColor) // down
    }
}
```

8-Connected Boundary Fill

```
BoundaryFill8(x, y, fillColor, boundaryColor)
{
    if getPixel(x, y) != boundaryColor AND getPixel(x, y) != fillColor
    {
        setPixel(x, y, fillColor)

        BoundaryFill8(x + 1, y, fillColor, boundaryColor) // right
        BoundaryFill8(x - 1, y, fillColor, boundaryColor) // left
        BoundaryFill8(x, y + 1, fillColor, boundaryColor) // up
        BoundaryFill8(x, y - 1, fillColor, boundaryColor) // down
        BoundaryFill8(x + 1, y + 1, fillColor, boundaryColor) // upper-right
        BoundaryFill8(x - 1, y + 1, fillColor, boundaryColor) // upper-left
        BoundaryFill8(x + 1, y - 1, fillColor, boundaryColor) // lower-right
        BoundaryFill8(x - 1, y - 1, fillColor, boundaryColor) // lower-left
    }
}
```

4-Connected Flood Fill

```
FloodFill4(x, y, oldColor, newColor)
{
    if getPixel(x, y) == oldColor
    {
        setPixel(x, y, newColor)

        FloodFill4(x + 1, y, oldColor, newColor) // right
        FloodFill4(x - 1, y, oldColor, newColor) // left
        FloodFill4(x, y + 1, oldColor, newColor) // up
        FloodFill4(x, y - 1, oldColor, newColor) // down
    }
}
```

8-Connected Flood Fill

```
FloodFill8(x, y, oldColor, newColor)
{
    if getPixel(x, y) == oldColor
    {
        setPixel(x, y, newColor)

        FloodFill8(x + 1, y, oldColor, newColor) // right
        FloodFill8(x - 1, y, oldColor, newColor) // left
        FloodFill8(x, y + 1, oldColor, newColor) // up
        FloodFill8(x, y - 1, oldColor, newColor) // down
        FloodFill8(x + 1, y + 1, oldColor, newColor) // upper-right
        FloodFill8(x - 1, y + 1, oldColor, newColor) // upper-left
        FloodFill8(x + 1, y - 1, oldColor, newColor) // lower-right
        FloodFill8(x - 1, y - 1, oldColor, newColor) // lower-left
    }
}
```